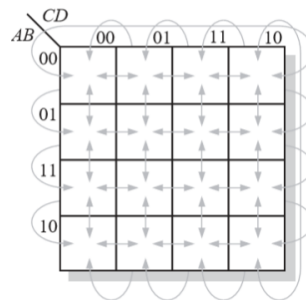


Mapas de Karnaugh

Proporciona un método sistemático de simplificación de expresiones booleanas. Es similar a una tabla de verdad, ya que muestra todos los valores posibles de las variables de entrada y la salida resultante para cada valor. Es una **matriz de celdas** en la que cada celda representa un valor binario de las variables de entrada. Las celdas se organizan de manera que la simplificación de una determinada expresión consiste en agrupar adecuadamente las celdas.

El **número de celdas** de un mapa de Karnaugh es igual al número total de posibles combinaciones de las variables de entrada.

Adyacencia de celdas: las celdas de un mapa de Karnaugh se disponen de manera que sólo cambia una única variable entre celdas adyacentes. Cada celda es adyacente a las celdas que están situadas inmediatas a ella por cualquiera de sus cuatro lados. Además, las celdas de la fila superior son adyacentes a las de la fila inferior y las celdas de la columna izquierda son adyacentes a las situadas en la columna de la derecha.

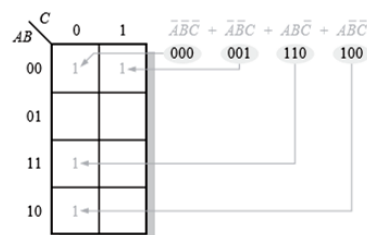


Utilidad: Reduce una expresión suma de productos, la cual está formada por el mínimo número de términos producto posibles con el mínimo número de variables por término. Generalmente, una expresión suma de productos minimizada puede implementarse mediante un número de puertas menor que su expresión estándar. Reducir el circuito a su suma de productos permite reducir el tiempo de retardo de propagación total.

También puede reducir producto de sumas si se representan los 0s en lugar de los 1s.

Conformar el mapa: el valor de cada celda se determina por el valor del producto entre los valores de las variables de entrada en la misma fila y columna de la celda. En la minimización de la suma de productos se colocan los 1s únicamente.

Si la suma de productos se encuentra en forma estándar, cosa que sucede cuando se utiliza la tabla de verdad para la configuración del mapa, cada expresión de la SOP se ve reflejada en el mapa. Esto significa que habrá tantos 1s como términos.



Armaz la mínima expresión: ya se dijo que la mínima expresión se obtiene de agrupar adecuadamente las celdas, en el caso de representar los 1s. La idea es maximizar el tamaño de los grupos y minimizar la cantidad de grupos. Los 1s se agrupan de la siguiente manera:

- Un grupo tiene que contener una cantidad de 1s que se corresponda con las potencias de 2.
- Cada celda de un grupo tiene que ser adyacente a una o más celdas del mismo grupo, pero no todas las celdas del grupo tienen que ser adyacentes entre sí.
- Incluir siempre en cada grupo el mayor número posible de 1s de acuerdo a la regla número 1.

- Cada 1 del mapa tiene que estar incluido en al menos un grupo. Los 1s que ya pertenezcan a un grupo pueden estar incluidos en otro, siempre que los grupos que se solapen contengan 1s no comunes.

Una vez agrupados los 1s, se procede a arupar las celdas que contienen 1s. Cada grupo de celdas que contiene 1s da lugar a un término producto compuesto por todas las variables que aparecen en el grupo en sólo una forma, esto es únicamente complementada, o no complementada. Si suceden ambas cosas, la variable se desecha y es una **variable contradictoria**. Finalmente se suman los términos producto obtenidos.

Condiciones indiferentes: surgen en las situaciones en las que algunas combinaciones de las variables de entrada no están permitidas, en esos casos los términos son indiferentes con respecto a su efecto en la salida, y podría asignárseles tanto un 0 como un 1. Una función que tiene al menos una condición indiferente se llama **función incompleta**. **Ejemplo** los números por encima del 9 en un código BCD.

Pueden utilizarse para aprovechar mejor el método del mapa de Karnaugh. Para cada término indiferente, se escribe una X en la celda. Cuando se agrupan los 1s, las X pueden ser consideradas también como 1s para agrandar los grupos, o como 0s si no obtenemos ninguna ventaja.

Mapa de 5 variables: se pueden hacer dos mapas de 4 variables, pero en cada uno predeterminar cada valor de la quinta variable. Se debe imaginar que uno de los mapas se encuentra directamente encima del otro para realizar las agrupaciones.

Método Tabular de Quine-McCluskey

GENERACIÓN DE IMPLICANTES PRIMOS

Los **implicantes primos** de una función lógica f son los cubos k más grandes posibles para los que $f = 1$.

Creamos una lista de vértices para los que $f = 1$ o es una condición no-importa. Los vértices se agrupan en conjuntos de filas con la misma cantidad de 1s.

Comparamos luego estos vértices por parejas para ver si pueden combinarse en cubos más grandes.

Si se tienen dos cubos idénticos en todas las variables (coordenadas) excepto en una para la que un cubo tiene el valor 0 y el otro tiene 1, entonces dichos cubos pueden combinarse en un cubo más grande.

Por ejemplo: la lista 1 tiene los minitérminos de la función o los no me importa. La lista 2 agrupa los cubos, y la lista 3 combina los cubos anteriores para obtener otros más grandes. Los cubos sin marca en cada lista son los implicantes primos de f .

Lista 1			Lista 2			Lista 3		
0	0 0 0 0	✓	0,4	0 x 0 0	✓	0,4,8,12	x x 0 0	
4	0 1 0 0	✓	0,8	x 0 0 0	✓			
8	1 0 0 0	✓	8,10	1 0 x 0				
10	1 0 1 0	✓	4,12	x 1 0 0	✓			
12	1 1 0 0	✓	8,12	1 x 0 0	✓			
11	1 0 1 1	✓	10,11	1 0 1 x				
13	1 1 0 1	✓	12,13	1 1 0 x				
15	1 1 1 1	✓	11,15	1 x 1 1				
			13,15	1 1 x 1				

Siguiendo lo anterior, los implicantes primos de F son

$$P = \{10x0, 101x, 110x, 1x11, 11x1, xx00\}$$

$$= \{p_1, p_2, p_3, p_4, p_5, p_6\}$$

DETERMINACIÓN DE UNA COBERTURA MÍNIMA

Es preciso elegir un subconjunto de costo mínimo que abarque todos los mintérminos.

- Se construye una tabla de cobertura de implicantes primos en la que haya una fila por cada implicante primo y una columna por cada mintermino que deba cubrirse.
- Se colocan marcas para indicar los minterminos cubiertos por cada implicante primo. Si hay una sola marca en alguna columna de la tabla de cobertura, entonces el implicante primo que cubre el mintermino de esa columna es **esencial** y ha de incluirse en la cobertura final.
- Se eliminan las filas correspondientes a los implicantes primos esenciales y las columnas cubiertas por ellos.
- Se aplica la **dominancia de fila** para reducir la tabla de cobertura. Un implicante domina a otro cuando cubren algún mintermino en común, pero el primero cubre otros más. El dominante se debe mantener, y la fila del otro debe eliminarse.
- La cobertura final se forma por los implicantes que sobreviven en la tabla.

Implicante primo	0	4	8	Mintérmino				10	11	12	13	15
$p_1 = 1\ 0\ x\ 0$			✓		✓							
$p_2 = 1\ 0\ 1\ x$					✓	✓						
$p_3 = 1\ 1\ 0\ x$										✓	✓	
$p_4 = 1\ x\ 1\ 1$								✓				✓
$p_5 = 1\ 1\ x\ 1$											✓	✓
$p_6 = x\ x\ 0\ 0$	✓	✓	✓							✓		

a) Tabla inicial de cobertura de implicantes primos

Implicante primo	Mintérmino			
	10	11	13	15
p_1	✓			
p_2	✓	✓		
p_3			✓	
p_4		✓		✓
p_5			✓	✓

b) Después de eliminar los implicantes primos esenciales

Implicante primo	Mintérmino			
	10	11	13	15
p_2	✓	✓		
p_4		✓		✓
p_5			✓	✓

c) Después de eliminar las filas dominantes

Y la cobertura final es:

$$C = \{p_2, p_5, p_6\}$$

$$= \{101x, 11x1, xx00\}$$

Y la función se representa como

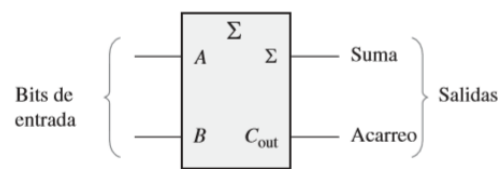
$$f = x_1\bar{x}_2x_3 + x_1x_2x_4 + \bar{x}_3\bar{x}_4$$

Circuitos combinacionales:

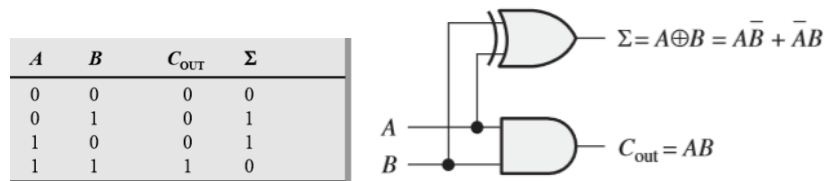
Cuando se conectan puertas lógicas entre sí, con el fin de generar una determinada salida específica para determinadas combinaciones específicas de las variables de entrada, sin que haya implicado almacenamiento, el circuito resultante se califica como **lógica combinacional**. En la lógica combinacional, el nivel de salida depende siempre de la combinación de los niveles de entrada.

Los siguientes son tipos de circuitos lógicos combinacionales:

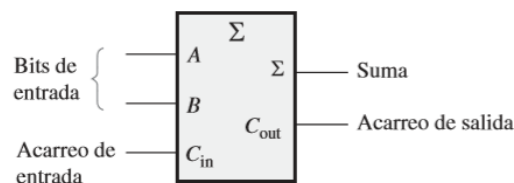
SEMISUMADOR: Un semi-sumador suma dos bits y genera una salida como bit de suma y una salida de acarreo. La expresión de la suma y el acarreo puede obtenerse como función de las dos entradas. La salida de acarreo es 1 solamente cuando A y B sean 1, por lo que puede expresarse como una operación AND entre las entradas. Y la salida de suma es 1 únicamente cuando las entradas son diferentes, por lo que se puede expresar como una operación XOR entre las entradas. A partir de estas dos operaciones se puede desarrollar la implementación lógica del funcionamiento de un semisumador:



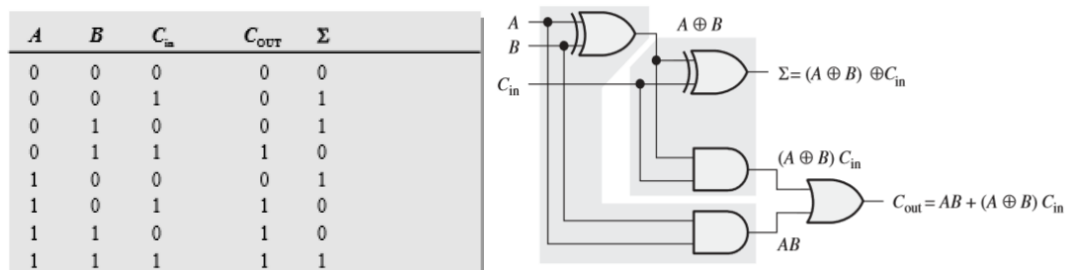
La tabla de verdad y el circuito lógico correspondiente se representan como:



SUMADOR COMPLETO: en este caso, se añade una entrada más que será el acarreo de entrada. Y las salidas son la suma y el acarreo de salida como en el semisumador.



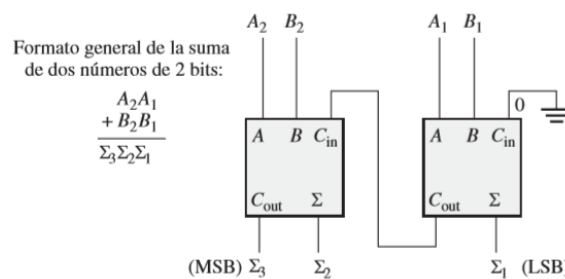
En este caso habrá que sumar el acarreo de entrada a las entradas mediante otra operación XOR. Por lo que la salida de la suma será $\Sigma = (A \oplus B) \oplus C_{in}$. El acarreo de salida es 1 cuando las dos entradas de la primera puerta XOR están a 1 o cuando las dos entradas de la segunda puerta XOR están a 1. Por lo que será $C_{out} = AB + (A \oplus B) C_{in}$



SUMADORES BINARIOS EN PARALELO: un único sumador completo es capaz de sumar dos números binarios de 1 bit y un acarreo de entrada. Para sumar números binarios de más de un bit, se tienen que utilizar sumadores completos adicionales. Cuando se suman dos números binarios, cada columna genera un bit de suma y un bit de acarreo, que se será el bit de acarreo de entrada del sumador inmediato de la izquierda. Estos se llaman **acarreo internos**. Se necesitan tantos sumadores completos como cantidad de bits se quieran sumar.

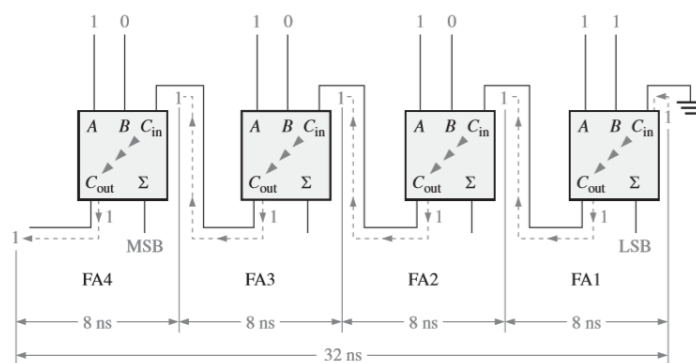
Se puede usar un semisumador para el bit menos significativo o bien poner a 0 la entrada del acarreo de entrada en un sumador completo, puesto que para el bit menos significativo no hay acarreo de entrada.

El acarreo de salida del sumador completo de más a la izquierda se convierte en el bit más significativo de la suma, es decir del resultado final.



En función del método utilizado para manipular los acarrees en un sumador paralelo, existen dos tipos: el **sumador de acarreo serie** y el **sumador de acarreo anticipado**. Ambos poseen las mismas entradas y salidas, pero difieren en la velocidad en que se realiza la suma.

- **SUMADOR DE ACARREO SERIE:** es más lento pero más simple a la vez. La salida de acarreo de cada sumador completo se conecta a la entrada de acarreo de la siguiente etapa de orden inmediatamente superior. La suma y el acarreo de salida de cualquier etapa no se pueden generar hasta que tiene lugar el acarreo de entrada, lo que da lugar a un retardo temporal en el proceso de adición, desde la aplicación del acarreo de entrada hasta que se produce el acarreo de salida.



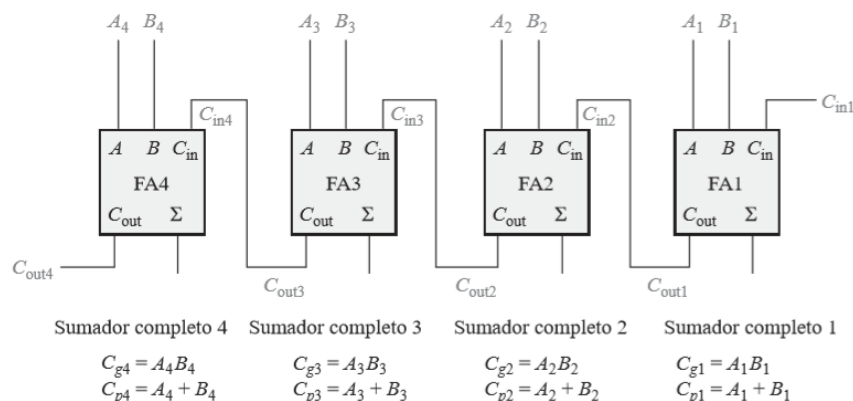
- **SUMADOR DE ACARREO ANTICIPADO:** anticipa el acarreo de salida de cada etapa y, en función de los bits de entrada de cada etapa, genera el acarreo de salida. El acarreo de salida se define combinando dos nuevos acarrees: un acarreo generado y un acarreo propagado. El **acarreo generado (C_g)** se expresa como la función AND de los 2 bits de entrada, A y B. El **acarreo propagado (C_p)** cuando uno o ambos bits de entrada son igual a 1, es decir un OR entre las entradas.

Obtenemos un acarreo de salida de 1 si el sumador completo los genera ($A=1$ AND $B=1$) o si el sumador propaga el acarreo de entrada ($A = 1$ OR $B = 1$) AND $C_{in}=1$.

Entonces, el acarreo de salida será

$$C_{out} = C_g + C_p C_{in}$$

De esta manera, Las funciones C_g y C_p para cada etapa están disponibles de forma *inmediata* tan pronto como se aplican los bits de entrada A y B y el acarreo de entrada del sumador menos significativo (LSB), ya que sólo dependen de estos bits, y no hay que esperar que se realice la suma.

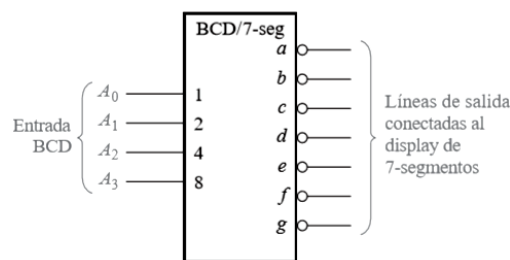


El único retardo que se produce es el paso por cada puerta.

Codificadores:

La función básica de un decodificador es detectar la presencia de una determinada combinación de bits (código) en sus entradas y señalar la presencia de este código mediante un cierto nivel de salida. En su forma general, un decodificador posee n líneas de entrada para gestionar n bits y en una de las 2^n líneas de salida indica la presencia de una o más combinaciones de n bits. **Ejemplo:**

- **Decodificador BCD a decimal** convierte cada código BCD (código 8421) en uno de los diez posibles dígitos decimales. Cada una de estas funciones se implementa mediante puertas NAND para proporcionar salidas activas a nivel BAJO.
- **Decodificador BCD a 7 segmentos:** acepta el código BCD en sus entradas y proporciona salidas capaces de excitar un display de 7-segmentos para generar un dígito decimal.



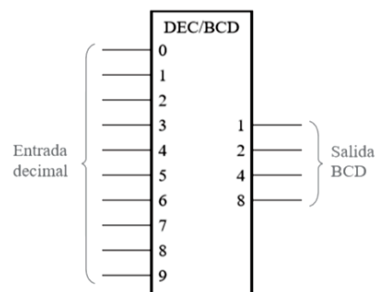
- **Decodificador 4-Hexa:** recibe como entradas combinaciones de 4 bits, y en la salida genera su correspondiente valor en sistema Hexadecimal reflejado en un 7 segmentos.
- **Hamming a binario.**

Decodificadores:

Un codificador es un circuito lógico combinacional que, esencialmente, realiza la función "inversa" del decodificador. Un codificador permite que se introduzca en una de sus entradas un nivel activo que representa un dígito, como puede ser un dígito decimal u octal, y lo convierte en una salida codificada, como BCD o binario.

Ejemplos:

- **Codificador decimal a BCD:** Este tipo de codificador tiene diez entradas, una para cada dígito decimal, y cuatro salidas que corresponden al código BCD.



Este codificador también se puede plantear con prioridad decimal. Esto es, el codificador producirá una salida BCD correspondiente al dígito decimal de entrada de más alto orden que se encuentre activo, e ignorará cualquier otra entrada de menor orden que esté activa.

Uso: codificador de teclado. Por ejemplo, los diez dígitos decimales del teclado de una computadora tienen que codificarse para poder ser procesados por el circuito lógico.

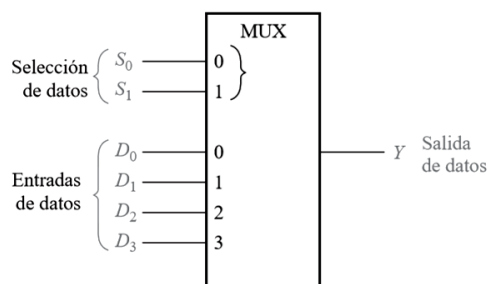
- **Binario a Hamming.**

Multiplexores (selectores de datos):

Circuito que conmuta los datos digitales desde una línea de entrada a varias líneas de salida según una secuencia temporal especificada.

Un multiplexor (MUX) es un dispositivo que permite dirigir la información digital procedente de diversas fuentes a una única línea para ser transmitida a través de dicha línea a un destino común. El multiplexor básico posee varias líneas de entrada de datos y una única línea de salida. También posee entradas de selección de datos, que permiten conmutar los datos digitales provenientes de cualquier entrada hacia la línea de salida.

Selector de datos: un código binario de dos bits en las entradas de selección de datos (S) va a permitir que los datos de la entrada (D) seleccionada pasen a la salida de datos. La salida de datos es igual al estado de la entrada de datos seleccionada. Por tanto, podemos deducir una expresión lógica para la salida en función de las entradas de datos y de las entradas de selección. Por lo tanto, se podrá hacer la tabla de verdad, minimizarla y realizar el circuito lógico correspondiente.



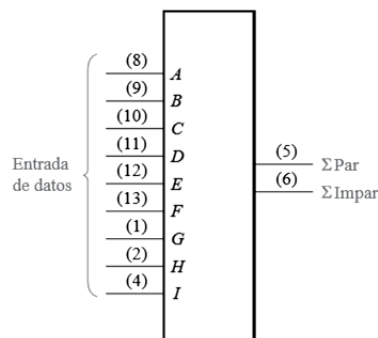
Demultiplexores (distribuidor de datos):

Circuito que conmuta los datos digitales de distintas líneas de entrada a una única línea de salida según una secuencia temporal especificada.

Un demultiplexor (DEMUX) básicamente realiza la función contraria a la del multiplexor. Toma datos de una línea y los distribuye a un determinado número de líneas de salida. Por esto se conoce también como distribuidor de datos. Como veremos, los decodificadores pueden utilizarse también como demultiplexores.

Generadores de paridad:

Generador/comprobador de paridad de 9 bits: Si este dispositivo se utiliza como generador de paridad par, el bit de paridad se toma en la salida Σ Impar, ya que esta salida es 0 cuando hay un número par de bits de entrada y 1 cuando hay un número impar. Cuando se emplea como generador de paridad impar, el bit de paridad se toma en la salida Σ Par, dado que ésta es 0 cuando el número de bits de entrada es impar.



La lógica que se usa para determinar la salida es: La suma OR exclusiva (descartando los acarreos) de un número par de 1s siempre es 0 y la suma de un número impar de 1s siempre es 1.

Aplicaciones:

El microprocesador Pentium realiza comprobaciones internas de paridad, así como comprobaciones de paridad de los buses externos de direcciones y datos. En una operación de lectura, el sistema externo puede transferir la información de paridad junto con los bytes de datos. El microprocesador Pentium comprueba si la paridad resultante es par y envía la señal correspondiente. Cuando se envía un código de dirección, este microprocesador no lleva a cabo una comprobación de paridad de la dirección, pero sí que genera un bit de paridad para la dirección.

Problemas y riesgos en el diseño:

Un **glitch** es un pico de tensión o de corriente (impulso) no deseado de muy corta duración. Los circuitos lógicos pueden interpretar estos impulsos como una señal válida, originando fallos en el funcionamiento del circuito.

Una manera de eliminar este problema es aplicar impulsos de validación (**strobing**), lo que consiste en activar el decodificador mediante un impulso de validación (strobe) únicamente durante los intervalos de tiempo en que las señales no se encuentran en un estado de transición